Backend · concept board

Django

MVT, ORM and migrations, views, DRF APIs, auth, admin, security

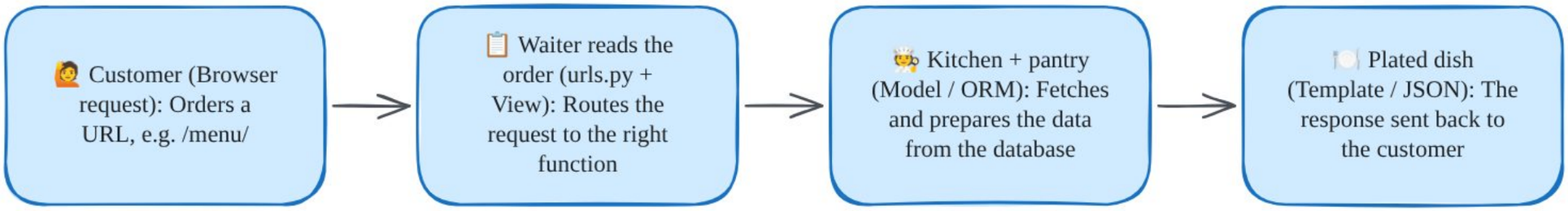
1. What Django is: the MVT flow
2. Models, ORM and migrations
3. URLs, views, forms and templates
4. Building APIs with Django REST framework
5. Auth, admin, security and deployment

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

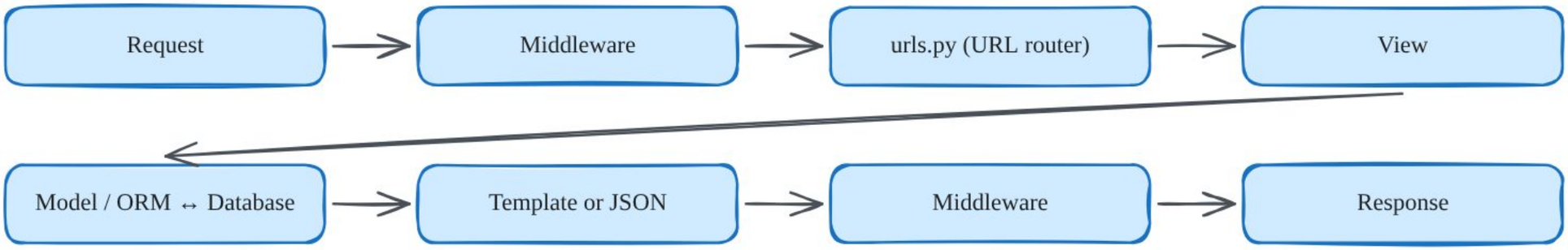
1. What Django is: the MVT flow

Definition

Django is a "batteries-included" Python web framework: ORM, migrations, admin, auth, forms, templates and security defaults in one package. It follows MVT: Model, View, Template.



Think of Django like a restaurant

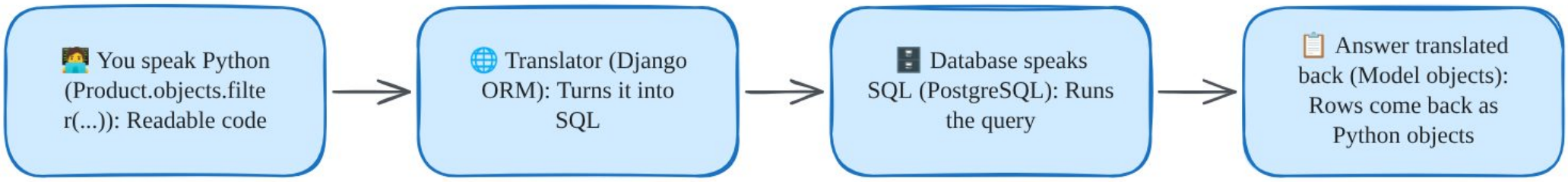


Start a project

```
python -m venv .venv && source .venv/bin/activate
pip install django
django-admin startproject shop .
python manage.py startapp products
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

- A project is the whole site (settings, root URLs); an app is one feature (products, users) you can reuse.
- Register apps in INSTALLED_APPS; configuration lives in settings.py (secrets and DEBUG come from environment variables).
- Django is synchronous by default; ASGI support allows async views and WebSockets (Channels).

2. Models, ORM and migrations



Think of the ORM like a translator at a meeting

products/models.py

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name="products")
    name = models.CharField(max_length=120)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    in_stock = models.BooleanField(default=True)
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created"]
        indexes = [models.Index(fields=["name"])]
# … 3 more lines in the notes
```

Querying with the ORM

```
from django.db.models import Count, Q

Product.objects.filter(in_stock=True, price__lt=500).order_by("name")
Product.objects.exclude(category__name="Toys").first()
Product.objects.filter(Q(name__icontains="pen") | Q(price__gt=1000))

# N+1 fix: join in one query instead of one query per row
for p in Product.objects.select_related("category"):       # ForeignKey / OneToOne
    print(p.name, p.category.name)
cats = Category.objects.prefetch_related("products")       # reverse FK / ManyToMany

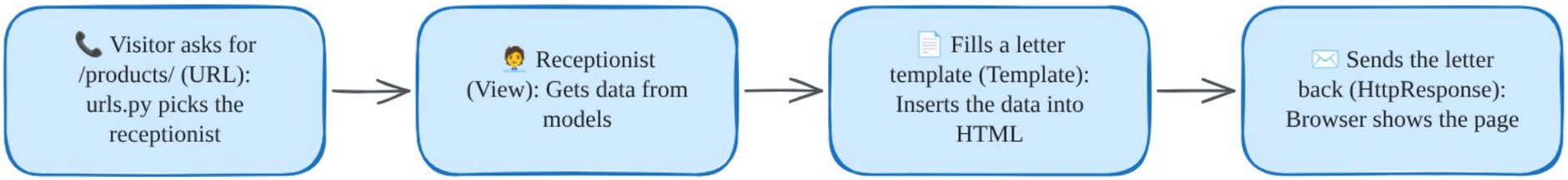
Category.objects.annotate(n=Count("products")).filter(n__gt=5)
```

- After changing models: makemigrations (writes the migration file) then migrate (applies it). Commit migration files.
- QuerySets are lazy: nothing hits the database until you iterate, slice or call list(), .count() or .exists().
- select_related (SQL JOIN) for single-valued relations and prefetch_related (extra query) for many-valued ones fix the N+1 problem.
- Wrap multi-step writes in transaction.atomic(). Use F() expressions for race-free updates (F("stock") - 1).

Common mistake

Looping over products and touching product.category inside the loop runs one extra query per row. Add select_related("category").

3. URLs, views, forms and templates



Think of a view like a receptionist

A URL, a view and a template

```
# products/urls.py
from django.urls import path
from . import views
urlpatterns = [
    path("", views.product_list, name="product-list"),
    path("<int:pk>/", views.ProductDetail.as_view(), name="product-detail"),
]

# products/views.py
from django.shortcuts import get_object_or_404, render
from django.views.generic import DetailView
from .models import Product

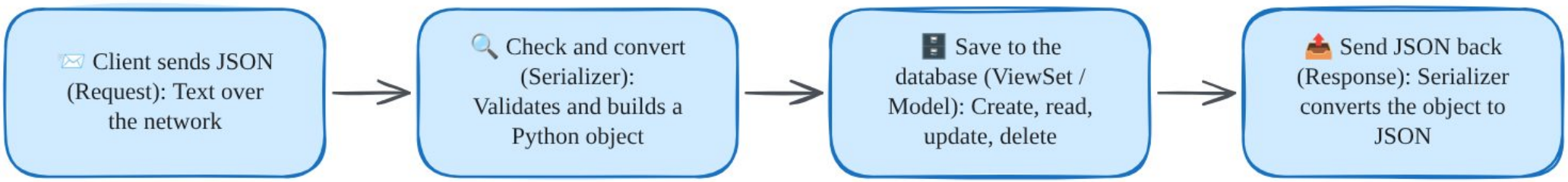
def product_list(request):                    # function-based view
    products = Product.objects.select_related("category")
# … 4 more lines in the notes
```

products/list.html (template language)

```
{% extends "base.html" %}
{% block content %}
  {% for p in products %}
    <h2>{{ p.name }} - ₹{{ p.price }}</h2>      <!-- auto-escaped against XSS -->
  {% empty %}
    <p>No products.</p>
  {% endfor %}
{% endblock %}
```

- FBV (function-based views) are simple and explicit; CBV (class-based) reuse behaviour through generic views like ListView, DetailView, CreateView.
- Forms / ModelForms validate input and render fields; {% csrf_token %} is required in POST forms.
- Middleware wraps every request (security, sessions, auth, CSRF, your own logging).
- Use reverse("product-detail", args=[1]) and {% url %} instead of hard-coding URLs.

4. Building APIs with Django REST framework



Think of DRF like a translator between JSON and objects

Serializer + ViewSet + router

```
from rest_framework import serializers, viewsets, permissions
from rest_framework.routers import DefaultRouter

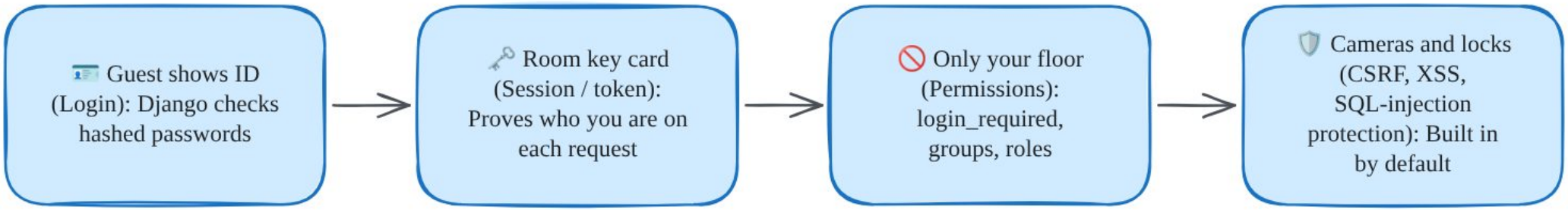
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "category"]

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("Price must be positive")
        return value

class ProductViewSet(viewsets.ModelViewSet):     # list, create, retrieve, update, delete
    queryset = Product.objects.select_related("category")
# … 6 more lines in the notes
```

- Serializers convert model instances to JSON and validate incoming data (like forms for APIs).
- ViewSets + routers generate the standard REST routes; add pagination_class, filters and throttling in settings.
- Authentication options: session, token, JWT (SimpleJWT). Permissions decide *who may do what*.
- Return the right status codes (201, 400, 401, 403, 404) and keep one consistent error shape.

5. Auth, admin, security and deployment



Think of Django security like a hotel front desk

- Built-in User model, login/logout views, password hashing (PBKDF2/Argon2), groups and permissions. Start with a custom user model (AUTH_USER_MODEL) in new projects.
- @login_required / LoginRequiredMixin protect views; @permission_required checks specific permissions.
- The admin (admin.site.register(Product)) gives you instant CRUD for staff: customise with ModelAdmin (list_display, search_fields).
- Built-in protections: ORM parameterises queries (SQL injection), templates auto-escape (XSS), CSRF middleware, clickjacking header, secure cookies.
- Production: DEBUG=False, set ALLOWED_HOSTS, keep SECRET_KEY in env vars, collectstatic + WhiteNoise/CDN, run with gunicorn/uvicorn behind Nginx, use PostgreSQL.
- Background work (emails, reports) goes to Celery + Redis, not inside the request.
- Test with django.test.TestCase or pytest-django; run python manage.py check --deploy before launch.

Revise

MVT flow, startproject vs startapp, models → makemigrations → migrate, lazy QuerySets, select_related/prefetch_related for N+1, FBV vs CBV, DRF serializers + ViewSets, CSRF/XSS/SQLi protections, DEBUG off, gunicorn behind Nginx, Celery for background jobs.

Rapid-fire Q&A (1/2)

What is the MVT pattern in Django?

Model (data via the ORM), View (logic that handles a request and returns a response), Template (HTML presentation). The URL router picks the view.

Project vs app in Django?

A project is the whole website with settings and root URLs; an app is a reusable module that implements one feature.

What are migrations?

Version-controlled files that describe changes to your models; makemigrations creates them and migrate applies them to the database.

What is the N+1 problem and how does Django fix it?

One query for the list plus one extra query per row for a relation. Use select_related (JOIN) or prefetch_related (batched query).

select_related vs prefetch_related?

select_related does a SQL JOIN for ForeignKey/OneToOne; prefetch_related runs a separate query and joins in Python, for ManyToMany and reverse relations.

Are QuerySets lazy?

Yes. The database is only hit when you evaluate the QuerySet (iterate, slice with a step, list(), len(), bool, .count()/.exists()).

Rapid-fire Q&A (2/2)

Function-based vs class-based views?

FBVs are explicit functions; CBVs use classes and mixins to reuse common behaviour such as list/detail/create pages.

What is middleware?

A hook that processes every request and response (security headers, sessions, auth, CSRF), configured in the MIDDLEWARE list in order.

How does Django prevent CSRF?

The CsrfViewMiddleware requires a secret token with unsafe methods (POST/PUT/DELETE); forms include {% csrf_token %}.

What is DRF and what is a serializer?

Django REST framework adds API tools. A serializer converts model instances to JSON and validates incoming data.

How do you deploy Django?

DEBUG off, ALLOWED_HOSTS and SECRET_KEY set, collectstatic, run gunicorn/uvicorn behind Nginx, PostgreSQL, and Celery for background tasks.

What do signals do?

They let decoupled code react to events such as post_save of a model; use sparingly because they hide control flow.